

OLIMPO — FAKE NEWS AI

Documentação de Arquitetura e Decisões de Engenharia

Visão Arquitetural Abrangente de Software:

Fundamentos, Padrões Estruturais, Persistência, Comunicação em Tempo Real, Segurança de Sessão e Organização Modular.

Projeto:	Plataforma Olimpo — Gamificação e Combate à Desinformação
Parcerias:	PUCC & Instituto de Pesquisas Eldorado
Stack Principal:	Next.js 16, TypeScript, PostgreSQL (Supabase / Drizzle), Redis, SSE
Tipografia:	Calibri (Sans-Serif Corporativo)
Versão do Doc:	1.0.0
Data:	2026

Sumário

1	OpenSpec	1
1.1	O que é?	1
1.2	Por que?	1
1.3	Como?	1
2	Arquitetura Backend	2
2.1	O que é?	2
2.2	Por que?	2
2.3	Como?	3
3	Banco de Dados e Cache	3
3.1	O que é?	3
3.2	Por que?	4
3.3	Como?	4
4	Supabase (Infraestrutura de Banco de Dados)	5
4.1	O que é?	5
4.2	Por que?	5
4.3	Como?	6
5	Next.js para Backend e Frontend	6
5.1	O que é?	6
5.2	Por que?	6
5.3	Como?	7
6	SSE para Tempo Real e Não WebSocket	7
6.1	O que é?	7
6.2	Por que?	7
6.3	Como?	8
7	Serverless	9
7.1	O que é?	9
7.2	Por que?	9
7.3	Como?	9

8	Respostas API	10
8.1	O que é?	10
8.2	Por que?	10
8.3	Como?	10
9	Autenticação	11
9.1	O que é?	11
9.2	Por que?	11
9.3	Como?	12
10	Dependency Injection	13
10.1	O que é?	13
10.2	Por que?	13
10.3	Como?	14
11	Layered Arch (Arquitetura em Camadas)	14
11.1	O que é?	14
11.2	Por que?	15
11.3	Como?	15
12	Validação de Dados	16
12.1	O que é?	16
12.2	Por que?	16
12.3	Como?	16
13	Padrão de Código	17
13.1	O que é?	17
13.2	Por que?	17
13.3	Como?	17
14	FSD (Feature-Sliced Design)	18
14.1	O que é?	18
14.2	Por que?	18
14.3	Como?	19
15	Testes de Software e Garantia de Qualidade	20

15.1 O que é?	20
15.2 Por que?	20
15.3 Como?	21
16 Considerações Finais	23

1 OpenSpec

1.1 O que é?

O **OpenSpec** é um padrão e framework de governança técnica para especificação viva de software baseada em capacidades (*Capability-Based Specifications*) e gestão estruturada de mudanças evolutivas (*Spec-Driven Development*). Em vez de documentações estáticas ou isoladas em wikis externas que rapidamente se tornam obsoletas, o OpenSpec reside diretamente dentro do repositório de código, na pasta raiz `/openspec`. Ele divide as definições do projeto em duas frentes complementares: especificações canônicas de capacidades permanentes da aplicação (`openspec/specs/`) e propostas de alteração incrementais e versionadas (`openspec/changes/`).

Cada especificação segue um modelo orientado ao comportamento (Behavior-Driven Development – BDD) e aos contratos de domínio, formalizando propósitos claros, requisitos inequívocos e cenários compostos pelas diretivas **WHEN** (quando) e **THEN** (então).

1.2 Por que?

A adoção do OpenSpec no projeto Olimpo visa solucionar três desafios críticos de engenharia de software contemporânea:

1. **Alinhamento entre Humanos e Agentes de IA:** A inteligência artificial colaborativa necessita de limites contratuais rígidos e contexto verificável para evitar alucinações de escopo e modificações não autorizadas em contratos de APIs já validados.
2. **Rastreabilidade Histórica de Decisões:** Muitas decisões de design são tomadas oralmente ou em mensagens efêmeras. O OpenSpec documenta a motivação (`proposal.md`), as decisões de arquitetura e trade-offs (`design.md`) e a lista atômica de tarefas (`tasks.md`) antes que qualquer linha de código seja modificada.
3. **Prevenção de Débito Técnico e Desvio de Escopo:** Nenhuma alteração entra no projeto sem que seus cenários de teste e critérios de aceitação estejam previamente aprovados e sincronizados com a especificação principal através do comando de `sync` ou `archive`.

1.3 Como?

No Olimpo, o OpenSpec é estruturado no diretório raiz conforme a árvore abaixo:

- `openspec/specs/`: Contém as capacidades estáveis consolidadas do sistema:
 - `auth/spec.md`: Especificação de cadastro, login com hash e emissão de JWT.
 - `rooms/spec.md`: Ciclo de vida de salas, código PIN (XXX XXX), lobbies e playlists.
 - `news-voting/spec.md`: Ciclo de votação dos jogadores em 3 níveis (*reliable*, *uncertain*, *unreliable*).

- `realtime-events/spec.md`: Transmissão contínua de eventos de sala via Server-Sent Events (SSE).
 - `ai-feedback/spec.md`: Análise explicativa pedagógica da veracidade das manchetes.
 - `gamification/spec.md`: Mecânicas de pontuação, bônus de blefe e ranking.
- `openspec/changes/`: Contém os deltas de mudanças em andamento e o histórico de mudanças arquivadas (`archive/`).
 - **Fluxo de Trabalho Operacional:**
 1. **Proposta:** O desenvolvedor inicia uma mudança com proposta de escopo e análise de impacto.
 2. **Design e Tarefas:** Redação do `design.md` com metas, não-metas e trade-offs técnicos, acompanhado de `tasks.md`.
 3. **Implementação e Validação:** Implementação orientada a testes unitários (TDD) e verificação com linters e compilador.
 4. **Sincronização e Arquivamento:** Atualização das especificações canônicas e transferência da proposta para a pasta de arquivo histórico.

2 Arquitetura Backend

2.1 O que é?

A arquitetura de backend do Olimpo segue o princípio de **Clean Architecture Adaptada e Orientada a Funcionalidades** (*Feature-First Layered Architecture*). Em contraste com arquiteturas em que o código de domínio se perde misturado a middlewares web ou ORMs acoplados, o Olimpo organiza as capacidades de negócio em módulos autocontidos dentro de `src/app/api/{feature}/`, apoiados por um núcleo compartilhado em `src/server/shared/`.

O design garante que as entidades e regras de negócio essenciais (como a lógica de cálculo de pontuação de blefe ou a validação matemática de PIN de sala) sejam totalmente agnósticas de framework, desconhecendo detalhes técnicos sobre HTTP, Next.js, Drizzle ou Redis.

2.2 Por que?

A escolha dessa arquitetura decorre dos seguintes fatores fundamentais:

- **Independência de Detalhes Tecnológicos:** Mecanismos de banco de dados, bibliotecas de hash ou provedores de mensageria podem ser substituídos sem alterar nenhuma linha da lógica do jogo.
- **Alta Testabilidade (TDD):** Casos de uso e entidades podem ser executados e testados em milissegundos utilizando dublês de teste (*mocks* ou repositórios em memória), sem exigir subida de contêineres ou instâncias reais de banco.

- **Coesão Vertical e Escalabilidade de Time:** Todo o ciclo de uma funcionalidade (entidade, caso de uso, repositório, action e testes) reside sob o mesmo diretório temático, evitando saltos cognitivos por pastas distantes.

2.3 Como?

A estrutura de pastas e responsabilidades é rigidamente padronizada:

- `src/app/api/{feature}/`:
 - `entities/`: Entidades de domínio puras e Value Objects com suas regras invariantes e validações intrínsecas (ex.: `RoomPin`, `UserEntity`).
 - `usecase/`: Classes de casos de uso (Application Services) contendo o fluxo orquestrador da regra de negócio (ex.: `CreateRoomUseCase`, `LoginUseCase`).
 - `repositories/`: Contratos de interface que definem o que a camada de domínio precisa (ex.: `IRoomRepository`), juntamente com as implementações de infraestrutura concretas baseadas em Drizzle ORM ou Redis.
 - `actions/`: Server Actions que servem como porta de entrada oficial para invocações do frontend, tratando deserialização de dados e validação de schema via Zod.
 - `tests/`: Baterias completas de testes unitários e de integração focadas na respectiva feature.
- `src/server/shared/`: Serviços de infraestrutura globais compartilhados:
 - `database/`: Cliente Drizzle singleton com `postgres.js` e schemas relacionais centrais.
 - `redis/`: Cliente singleton `ioredis` com suporte a reuso de conexões em ambientes serverless.
 - `logger/`: Sistema de logs estruturados em JSON via Pino.

Fluxo de Execução Unidirecional:

UI / Formulário → Server Action (Zod) → Use Case → Repository Interface ← Drizzle / Redis

3 Banco de Dados e Cache

3.1 O que é?

Trata-se de uma estratégia de persistência poliglota e híbrida que combina um banco de dados relacional de conformidade ACID (**PostgreSQL**, hospedado no **Supabase**) com um armazenamento em memória de altíssimo desempenho (**Redis**). No Olimpo, o PostgreSQL é gerenciado pelo **Drizzle ORM** (com o driver nativo `postgres.js`), enquanto o cache, controle de conexões de eventos e dados efêmeros de partidas em andamento são gerenciados pelo **ioredis**.

3.2 Por que?

Uma aplicação gamificada em tempo real apresenta padrões de acesso diametralmente opostos:

- **Dados Duráveis e Relacionais:** Informações como cadastros de usuários, credenciais, histórico de partidas concluídas, acervo de notícias e seus metadados estruturados exigem garantias transacionais rígidas, integridade referencial com chaves estrangeiras e durabilidade permanente. O PostgreSQL é excelente nesse papel.
- **Dados Efêmeros e Alta Frequência:** Consultas de código PIN de sala ativa, apuração instantânea de votos a cada rodada cronometrada, ranking dinâmico de participantes e canais de mensageria Pub/Sub geram um volume massivo de leituras e escritas por segundo. Manter essa carga exclusivamente no PostgreSQL causaria gargalos de concorrência, contenção de locks e degradação de I/O em disco. O Redis opera integralmente em memória RAM, oferecendo latência sub-milissegundo para o estado ativo da sala.

3.3 Como?

A integração de dados no projeto Olimpo ocorre da seguinte maneira:

1. PostgreSQL com Drizzle ORM:

- Definição de schemas tipados com TypeScript em `src/server/shared/database/schemas/`.
- Uso estratégico da coluna `article` tipada como **JSONB** na tabela `news_articles`, permitindo armazenar o documento extraído pelo motor de ingestão (título, autor, fonte, trecho, imagens) sem necessidade de esparsidade ou migrações destrutivas.
- Enums nativos para integridade estrita: `ml_target_type` e `vote_option_type` contendo ('reliable', 'uncertain', 'unreliable').
- Migrações versionadas e determinísticas geradas com o utilitário `drizzle-kit`.

2. Redis (ioredis):

- **Lookup O(1) de PIN:** Chave `room:pin:<pin>` associando o código alfanumérico ao ID da sala e status atual, viabilizando entrada rápida sem consulta relacional prévia.
- **Placar Dinâmico (Sorted Sets):** Uso dos comandos atômicos `ZADD` e `ZREVRANGE` na chave `room:<id>:scores` para manter o ranking ordenado de pontuações de forma automática e imediata.
- **Canais Pub/Sub:** Publicação de eventos na chave de canal `room:<pin>` para disparar atualizações para ouvintes SSE.
- **Padrão Singleton Seguro:** Implementação em `client.ts` que preserva a conexão em `globalThis` durante o ciclo de recarregamento a quente (*Hot Module Reload*) em ambiente de desenvolvimento.

4 Supabase (Infraestrutura de Banco de Dados)

4.1 O que é?

O **Supabase** é uma plataforma de código aberto baseada em computação em nuvem que fornece um ecossistema completo de *Backend-as-a-Service* (BaaS) estruturado sobre o motor relacional **PostgreSQL**. Na arquitetura da plataforma Olimpo, o Supabase foi selecionado e configurado como a infraestrutura de hospedagem e gestão em nuvem do banco de dados relacional. Ele disponibiliza instâncias corporativas de PostgreSQL com alta disponibilidade, serviços nativos de pooler de conexões transacionais (**Supavisor**), backups contínuos automatizados, observabilidade em tempo real e um console administrativo visual (Supabase Studio) para auditoria e governança técnica.

4.2 Por que?

A opção por implementar a base relacional no Supabase, em vez de recorrer a soluções manuais autogerenciadas ou outros bancos legados, fundamenta-se nos seguintes pilares de engenharia:

- **Sobrecarga Operacional Nula (Zero Ops):** A equipe de desenvolvimento não despende tempo gerenciando sistemas operacionais de servidores dedicados, configurando volumes de disco virtual (EBS), executando rotinas manuais de recuperação de desastres ou aplicando patches de vulnerabilidade no daemon do PostgreSQL. O ciclo operacional é plenamente gerenciado pela nuvem.
- **Connection Pooling para o Paradigma Serverless:** A execução de rotas e procedimentos no Next.js (por meio de Server Actions e Route Handlers sob demanda) opera em ciclos de vida efêmeros. O acesso concorrente direto de funções serverless tradicionais ao PostgreSQL causaria rápida exaustão de conexões (*connection exhaustion*). O Supabase resolve nativamente essa barreira com o **Supavisor** no modo transação (porta 6543), permitindo que centenas de invocações paralelas compartilhem um pool controlado de conexões seguras sem sobrecarregar o banco.
- **Desacoplamento e Independência de Fornecedor (Sem Vendor Lock-in):** Embora o banco de dados resida na infraestrutura do Supabase, o Olimpo conecta-se estritamente como um cliente padrão PostgreSQL através do **Drizzle ORM** e do driver `postgres.js`. O vínculo dá-se unicamente via string de conexão (`DATABASE_URL`), sem qualquer acoplamento a bibliotecas proprietárias de cliente do Supabase no núcleo da aplicação. Caso haja necessidade futura, a base de dados pode ser migrada para qualquer outro provedor PostgreSQL sem modificar uma única linha dos casos de uso ou repositórios.
- **Console Studio e Governança de Dados:** O painel administrativo integrado possibilita inspecionar tabelas, monitorar a latência de consultas analíticas, verificar estatísticas de cache e validar a execução correta de transações diretamente na nuvem.

4.3 Como?

A implementação e o ciclo de vida da persistência com o Supabase no Olimpo são estruturados da seguinte maneira:

1. Estratégia de Conexão com Pooling e Singleton:

- A conexão é provisionada pela variável de ambiente `DATABASE_URL` em `.env`, aplicando obrigatoriamente criptografia TLS/SSL em trânsito com `sslmode=require`.
- Em tempo de execução da aplicação web (`src/server/shared/database/client.ts`), o cliente instancia o driver com `postgres(connectionstring, { max: 1 })` apontando para a porta do pooler do Supabase, associando a instância ao `globalThis` para reuso inteligente entre invocações ativas em ambientes serverless.
- Em rotinas de migração de esquema, utiliza-se a conexão direta (porta 5432) para viabilizar comandos DDL estruturais e travas transacionais de integridade.

2. Migrações Declarativas com Drizzle Kit:

- Todos os esquemas de dados relacionais (`user_accounts`, `news_articles`, `rooms`, `news_votes`), tipos enumerados (`ml_target_type`, `vote_option_type`) e índices são definidos em TypeScript em `src/server/shared/database/schemas/`.
- A ferramenta `drizzle-kit` gera scripts SQL determinísticos aplicados diretamente contra a instância PostgreSQL no Supabase, garantindo paridade absoluta de ambiente e histórico versionado no Git.

3. Aproveitamento de Capacidades Avançadas do PostgreSQL:

- Armazenamento de notícias completas na coluna semiestruturada **JSONB** da tabela `news_articles`, viabilizando flexibilidade de metadados extraídos pelo motor de ingestão sem perda da rigidez relacional.
- Chaves primárias baseadas em UUID v4 geradas nativamente e restrições de integridade referencial com remoção em cascata (`ON DELETE CASCADE`).

5 Next.js para Backend e Frontend

5.1 O que é?

Consiste na utilização do framework unificado **Next.js 16** (baseado em React 19) como uma solução integrada full-stack. O Next.js opera simultaneamente como plataforma de renderização do cliente e servidor de apresentação (com Server Components e Client Components) e como motor de execução de backend robusto através de rotas HTTP (**Route Handlers**) e procedimentos mutáveis remotos (**Server Actions**) marcados com a diretiva `"use server"`.

5.2 Por que?

A consolidação em uma arquitetura monorrepositório full-stack com Next.js oferece vantagens competitivas de engenharia:

- **Segurança de Tipos Fim-a-Fim (*End-to-End Type Safety*):** Interfaces de domínio, DTOs de entrada e saída e esquemas de validação Zod são compartilhados diretamente entre a tela do usuário e o caso de uso no servidor sem necessidade de geração de código intermediário ou contratos desincronizados.
- **Eliminação de Complexidade Operacional:** Elimina a necessidade de gerenciar múltiplos pipelines de implantação, múltiplos repositórios Git e negociações de CORS (Cross-Origin Resource Sharing), já que frontend e backend operam na mesma origem.
- **Otimização de Desempenho:** A utilização de React Server Components (RSC) reduz substancialmente o pacote de JavaScript enviado ao navegador do usuário, deixando tarefas pesadas de parsing DOM e cálculos de regras de negócio exclusivamente no servidor.

5.3 Como?

A estrutura de distribuição no projeto é particionada da seguinte forma:

- **Backend via Server Actions:** Em vez de expor dezenas de endpoints REST tradicionais para mutações simples, operações como autenticação, criação de sala e submissão de votos são funções assíncronas assinaladas com "use server". Exemplo: `loginAction`, `createRoomAction`, `submitVoteAction`.
- **Backend via Route Handlers:** Rotas HTTP tradicionais são reservadas para endpoints que necessitam de protocolos específicos de streaming contínuo ou webhooks externos, como a rota `GET /api/rooms/[pin]/events` para o fluxo de Server-Sent Events.
- **Isolamento de Segurança com `server-only`:** Módulos críticos de infraestrutura (como conexões de banco de dados e segredos de criptografia) importam obrigatoriamente o pacote "server-only". Se qualquer desenvolvedor tentar importar acidentalmente esses arquivos em componentes clientes, o compilador interrompe o build imediatamente.

6 SSE para Tempo Real e Não WebSocket

6.1 O que é?

A adoção de **Server-Sent Events (SSE)** representa a escolha de um protocolo web unidirecional padronizado pela W3C (baseado no tipo de mídia `text/event-stream`) sobre HTTP padrão para transmitir notificações e atualizações de estado do servidor para os clientes em tempo real, em detrimento do protocolo bidirecional **WebSocket**.

6.2 Por que?

A decisão de optar por SSE em vez de WebSockets no Olimpo foi pautada por rigorosos critérios arquiteturais e de custo de infraestrutura:

1. **Natureza Unidirecional da Interação:** Na dinâmica do Olimpo, as ações dos jogadores (entrar na sala, submeter voto na notícia, avançar rodada) são operações de mutação pontuais que exigem autenticação, validação de schema e persistência. Elas se encaixam perfeitamente no modelo de requisição HTTP (Server Actions). Não há necessidade de canal persistente do cliente para o servidor; apenas o servidor precisa empurrar dados contínuos para os jogadores (jogadores conectados, início de contagem regressiva, revelação do gabarito e feedback de IA).
2. **Compatibilidade com Arquitetura Serverless e HTTP/2:** WebSockets exigem servidores *stateful* com conexões TCP abertas e processos rodando ininterruptamente (ou serviços externos caros como Pusher ou AWS API Gateway WebSocket). O SSE opera nativamente sobre HTTP/2 e HTTP/3, aproveitando a multiplexação de conexões sem sobrecarregar a infraestrutura.
3. **Reconexão Automática e Simplicidade no Cliente:** A API padrão do navegador EventSource possui mecanismo nativo de reconexão automática com preservação do último evento recebido (*Last-Event-ID*), eliminando bibliotecas pesadas de cliente (como Socket.io) e reduzindo a superfície de bugs de rede.
4. **Transparência em Redes Corporativas e Proxies:** WebSockets frequentemente sofrem bloqueios por firewalls corporativos, proxies intermediários ou inspeções de segurança que não reconhecem o cabeçalho Upgrade. O SSE transita como tráfego HTTP comum via porta 443/TLS.

6.3 Como?

A implementação no Olimpo envolve duas pontas:

1. Servidor (`src/app/api/rooms/[pin]/events/route.ts`):

- Expõe uma rota GET com os cabeçalhos:

```
Content-Type: text/event-stream; charset=utf-8
Cache-Control: no-cache, no-transform
Connection: keep-alive
X-Accel-Buffering: no
```
- Instancia um cliente `ioredis` dedicado como assinante (*subscriber*) do canal `room:<pin>`.
- Utiliza a API web nativa `ReadableStream`. Sempre que uma mensagem chega ao canal Redis, o controlador a codifica e envia ao fluxo HTTP no formato event: `<tipo>\n data: <json>\n\n`.
- **Gerenciamento de Ciclo de Vida e Desconexão:** Escuta o evento `request.signal.addEvent`. Quando o usuário fecha a aba ou perde conexão, o listener remove as inscrições e encerra a conexão do Redis com `quit()`, evitando vazamento de memória (*connection leaks*).

2. Cliente (Frontend):

- O hook ou componente React conecta via `const es = new EventSource('/api/rooms/' + pin + '/events')`.

- Adiciona ouvintes tipados para cada evento do jogo: `MEMBER_JOINED`, `VOTE_CAST`, `ROUND_COMPLETED`, `MATCH_FINISHED`.

7 Serverless

7.1 O que é?

O modelo de computação **Serverless** (Computação Sem Servidor) refere-se à execução de código de backend em ambientes efêmeros, acionados por eventos e funções sob demanda (ex.: Vercel Serverless Functions ou AWS Lambda), onde o desenvolvedor não gerencia, monitora ou realiza manutenção em sistemas operacionais ou instâncias virtuais permanentes.

7.2 Por que?

A aplicação Olimpo foi projetada tendo o paradigma Serverless como horizonte estratégico:

- **Custo Proporcional à Utilização (Zero Idle Cost):** Plataformas de quiz e treinamento corporativo possuem tráfego oscilante; há períodos de pico durante treinamentos e aulas e longos períodos de inatividade. Em arquitetura serverless, paga-se estritamente pelos milissegundos de processamento ativo.
- **Elasticidade Instantânea:** Caso um evento simultâneo com centenas de salas e milhares de estudantes seja iniciado, a infraestrutura escala horizontalmente de forma automática sem intervenção manual de infraestrutura.
- **Foco Total no Produto:** Redução do custo operacional de manutenção de servidores, atualizações de segurança do sistema operacional e balanceadores de carga dedicados.

7.3 Como?

Para operar de forma robusta no modelo Serverless sem sofrer com problemas clássicos de *cold start* e esgotamento de conexões, o Olimpo implementa técnicas específicas:

- **Pool de Conexões com Reuso em `globalThis`:** As instâncias de conexão com PostgreSQL (`postgres.js`) e Redis (`ioredis`) são associadas ao escopo global do processo Node.js. Isso garante que, quando uma função lambda é mantida aquecida (*warm execution*), a conexão existente é reaproveitada, impedindo que milhares de conexões zumbis saturem o banco de dados.
- **Declaração de Runtime Node.js:** Definição explícita de `export const runtime = "nodejs"` e `export const dynamic = "force-dynamic"` nas rotas de API com SSE para assegurar compatibilidade plena com os módulos de rede e criptografia.
- **Execuções Stateless:** Nenhum estado mutável de sessão é armazenado na memória da máquina ou em disco local temporário; todo estado transita via JWT assinado nos cookies ou reside no Redis/PostgreSQL.

8 Respostas API

8.1 O que é?

É o padrão formal e padronizado de estruturação de mensagens de retorno trafegadas entre o backend e os consumidores (frontend ou clientes externos), contemplando contratos uniformes tanto para invocações de Server Actions quanto para endpoints HTTP REST. Esse formato utiliza tipos de união discriminada (*Discriminated Unions*) em TypeScript para modelar sucesso e falha.

8.2 Por que?

A ausência de padronização nas respostas de rede gera inconsistência, vulnerabilidades e código defensivo redundante:

- **Eliminação de Tratamento Caótico de Erros:** Impede cenários onde uma rota retorna `{ msg: "erro" }`, outra `{ message: "falha" }` e outra dispara exceções não tratadas com stack traces internas em HTML.
- **Tipagem Segura e Previsível no Frontend:** O compilador TypeScript exige que o desenvolvedor do frontend verifique explicitamente `if (result.success)` antes de tentar acessar os dados de retorno, prevenindo erros de `undefined is not an object` em produção.
- **Segurança e Prevenção de Vazamento de Dados:** Erros inesperados de infraestrutura são interceptados por blocos de captura centralizados, mascarando detalhes internos do banco de dados e expondo mensagens amigáveis ao usuário.

8.3 Como?

O padrão de respostas no Olimpo é dividido conforme o mecanismo de transporte:

1. **Contrato em Server Actions:** Todas as actions retornam um tipo genérico discriminado:

```
type ActionResult<T> =  
  | { success: true; data: T }  
  | { success: false; error: string; code?: string };
```

Exemplo real em `login.action.ts`:

```
export type LoginActionResult =  
  | { success: true; user: UserDTO }  
  | { success: false; error: string };
```

2. **Contrato em Route Handlers (HTTP REST):**

- Respostas de sucesso emitem status semânticos adequados (200 OK, 201 Created) acompanhadas do payload JSON estruturado: { success: true, data: [...] }.
- Respostas de erro utilizam os códigos de status HTTP apropriados:
 - **400 Bad Request:** Falha de validação de schema Zod ou argumentos inválidos.
 - **401 Unauthorized:** Ausência de token JWT válido ou sessão expirada.
 - **404 Not Found:** Sala, notícia ou usuário inexistente.
 - **409 Conflict:** Tentativa de registrar e-mail duplicado ou entrar em sala já finalizada.
 - **500 Internal Server Error:** Falha inesperada tratada pelo logger central.

9 Autenticação

9.1 O que é?

A **Autenticação** no Olimpo é o sistema de identificação, credenciamento, controle de acesso e gerenciamento de sessões de usuários. A arquitetura adota um modelo baseado em credenciais próprias (e-mail e senha) com armazenamento criptográfico irreversível de senhas via `bcryptjs`, emissão e validação de **JSON Web Tokens (JWT)** assinados digitalmente com a biblioteca moderna `jose`, e persistência de sessão de alta segurança através de cookies gerenciados pelo servidor (*Server-Side Cookies*) com as flags `httpOnly`, `Secure` e `SameSite=lax`.

Diferente de sistemas de sessão *stateful* em que cada requisição exige uma consulta ao banco de dados para checar tabelas de sessões ativas, o Olimpo implementa autenticação *stateless*: o token JWT carrega em sua carga útil criptografada a identidade do usuário e suas declarações essenciais (*claims*), permitindo validação imediata em milissegundos sem I/O de rede desnecessário.

9.2 Por que?

A estratégia de autenticação foi projetada para sanar vulnerabilidades críticas e atender aos requisitos de alta concorrência da plataforma gamificada:

- **Segurança Contra Roubo de Sessão e XSS (*Cross-Site Scripting*):** Armazenar tokens de autenticação em `localStorage` ou `sessionStorage` permite que qualquer script malicioso leia o token diretamente. O uso de cookies protegidos com `httpOnly` impede que qualquer código JavaScript no navegador tenha acesso ao token de sessão, neutralizando esse vetor de ataque.
- **Proteção Contra CSRF (*Cross-Site Request Forgery*):** A flag `sameSite: "lax"` combinada com a invocação das mutações via Server Actions nativas do Next.js anula a possibilidade de comandos forjados a partir de sites terceiros.
- **Criptografia de Senhas Segura contra Ataques de Dicionário:** O uso de `bcryptjs` com fator de custo de 10 rounds de salt assegura que, mesmo em caso improvável de

vazamento da base de dados relacional, as senhas originais dos participantes não possam ser revertidas via ataques de força bruta ou tabelas pré-computadas (*rainbow tables*).

- **Escalabilidade e Desempenho Serverless:** Por ser *stateless*, a autenticação baseada em JWT não exige criação, leitura ou deleção de registros de sessão no PostgreSQL a cada Server Action invocada por jogadores no meio de uma partida, preservando as conexões do banco de dados para a dinâmica do jogo.
- **Prevenção contra Enumeração de Usuários:** Mensagens de erro de autenticação são intencionalmente opacas (ex.: "Credenciais inválidas"), impedindo que atores mal-intencionados descubram se determinado e-mail já está cadastrado no sistema.

9.3 Como?

A arquitetura de autenticação é encapsulada na funcionalidade modular `src/app/api/auth/` e orquestrada da seguinte forma:

1. **Entidade Rica de Domínio (`entities/user.entity.ts`):** A classe `UserEntity` encapsula a lógica pura de usuário e suas invariantes:

- `hashPassword(password)`: Gera o hash criptográfico assíncrono com salt de 10 rounds.
- `verifyPassword(password, hash)`: Compara a senha informada com o hash persistido.
- `validateEmail(email)`: Valida a conformidade de formato de e-mail com regex rigoroso.
- `toDTO()`: Serializa o usuário para o formato seguro `UserDTO` (`id`, `email`, `name`, `xp`, `createdAt`), garantindo que o hash de senha jamais seja exposto para camadas externas ou para a interface.

2. **Assistente Criptográfico e Gestão de Cookies (`entities/jwt.helper.ts`):** Utiliza a biblioteca de criptografia `jose`:

- `signSessionToken(payload)`: Gera um JWT assinado com algoritmo HS256 utilizando a chave secreta de ambiente `AUTH_SECRET`, com tempo de vida de 7 dias e payload contendo `{ sub: userId, email, name }`.
- `verifySessionToken(token)`: Deserializa e valida a assinatura e expiração do token de forma criptográfica.
- `getAuthCookieOptions()`: Centraliza as diretivas de proteção:

```
{ httpOnly: true, secure: process.env.NODE_ENV === "production",
  sameSite: "lax", path: "/", maxAge: 60 * 60 * 24 * 7 }
```

3. **Casos de Uso Desacoplados (`usecase/`):**

- `RegisterUseCase`: Normaliza o e-mail, verifica se já existe na base através de `IUserRepository`, gera o hash da senha, persiste o novo registro com pontuação inicial de 0 XP e retorna a entidade.

- **LoginUseCase:** Localiza o usuário por e-mail, compara o hash da senha via `UserEntity.verifyPassword` e gera o token JWT assinado.
 - **LogoutUseCase:** Coordena o fluxo de encerramento da sessão ativa.
 - **GetSessionUseCase:** Extrai e valida a identidade do usuário logado a partir do token.
4. **Server Actions e Interação com Cookies (actions/):** As ações expostas para o frontend utilizam a API nativa `cookies()` do Next.js (`next/headers`):
- **loginAction:** Valida a entrada com Zod, invoca `LoginUseCase` e grava o cookie:

```
const cookieStore = await cookies();
cookieStore.set(AUTH_COOKIE_NAME, token, getAuthCookieOptions());
```
 - **logoutAction:** Remove o cookie do navegador de forma definitiva:

```
const cookieStore = await cookies();
cookieStore.delete(AUTH_COOKIE_NAME);
```
5. **Resolução de Sessão Protegida (getCurrentUserSession):** Helper reutilizável consumido por Server Actions de outras funcionalidades do Olimpo (como criação de salas, início de partidas e submissão de votos na notícia). O helper recupera o cookie da requisição, valida o JWT e injeta o objeto `UserDTO` no caso de uso, rejeitando imediatamente requisições não autenticadas.

10 Dependency Injection

10.1 O que é?

A **Injeção de Dependência** (*Dependency Injection* – DI) é um padrão de projeto de software que concretiza o Princípio da Inversão de Dependência (a letra "D" do acrônimo SOLID). Consiste em fornecer a uma classe ou serviço as suas dependências externas (como clientes de banco de dados, repositórios de dados ou serviços de mensageria) a partir de um agente exterior, geralmente através dos parâmetros de seu método construtor, em vez de permitir que a própria classe crie diretamente suas instâncias concretas.

10.2 Por que?

A aplicação rigorosa de Injeção de Dependência viabiliza benefícios indispensáveis:

- **Desacoplamento Arquitetural:** Os casos de uso de negócio conhecem apenas contratos abstratos (interfaces TypeScript), não sabendo se os dados vêm de uma tabela PostgreSQL, de um documento JSONB, do Redis ou de uma API externa.
- **Testabilidade Extrema (Zero-Database Unit Testing):** Permite testar integralmente fluxos de negócio complexos em milissegundos injetando repositórios em memória ou mocks nos testes com Vitest, sem necessidade de levantar serviços externos ou realizar conexões lentas de rede.

- **Facilidade de Substituição e Manutenção:** Se a tecnologia de persistência for modificada, basta implementar uma nova classe que cumpra a interface correspondente, sem tocar em uma única linha dos casos de uso.

10.3 Como?

O Olimpo utiliza o padrão de injeção via construtor com valores padrão estratégicos para balancear testabilidade com facilidade de consumo:

1. Definição do Contrato Abstrato (Interface):

```
export interface IUserRepository {
  findByEmail(email: string): Promise<UserEntity | null>;
  create(user: UserEntity): Promise<UserEntity>;
}
```

2. Injeção no Construtor do Caso de Uso:

```
export class LoginUseCase {
  constructor(
    private readonly userRepository: IUserRepository = drizzleUserRepository
  ) {}
  async execute(input: LoginInput): Promise<LoginOutput> { ...
  }
}
```

3. Consumo em Produção e em Testes:

- **Em Produção:** A Server Action simplesmente invoca `new LoginUseCase()`, assumindo automaticamente o repositório real baseado em Drizzle/Postgres.
- **Em Testes Unitários:** A suíte de testes instancia `new LoginUseCase(mockUserRepository)`, controlando as respostas com precisão e testando cenários de erro, credenciais inválidas e conflitos de forma isolada e determinística.

11 Layered Arch (Arquitetura em Camadas)

11.1 O que é?

A **Arquitetura em Camadas** (*Layered Architecture*) é um padrão estrutural que organiza o sistema em camadas concêntricas e hierárquicas com responsabilidades bem delimitadas, estabelecendo uma **Regra de Dependência Unidirecional**: o código de camadas superiores pode depender de camadas inferiores, mas as camadas mais internas jamais podem conhecer ou importar elementos das camadas externas.

11.2 Por que?

Sistemas que misturam consultas a banco de dados diretamente dentro de componentes visuais ou handlers de requisição tornam-se insustentáveis à medida que o escopo cresce:

- **Isolamento de Responsabilidades:** Modificações de layout no frontend ou alterações no driver de banco de dados não afetam as regras matemáticas e o domínio do jogo.
- **Manutenibilidade a Longo Prazo:** Qualquer membro da equipe sabe exatamente onde encontrar ou adicionar um código: entidades em `entities`, lógica em `usecase`, banco em `repositories` e endpoints em `actions`.
- **Clareza nos Contratos:** Garante que a camada de domínio permaneça pura, contendo apenas TypeScript puro sem dependências pesadas de bibliotecas de terceiros.

11.3 Como?

No Olimpo, as camadas são organizadas por funcionalidade (*Feature-First*):

- **1. Camada de Domínio (`entities/`):**
 - Contém entidades ricas, enums e Value Objects.
 - Responsável pelas invariantes de negócio e validações puras (ex.: verificar se um PIN tem 6 dígitos, hash e conferência de senhas).
 - **Regra de Ouro:** Dependência zero. Não importa Next.js, React, Drizzle ou bancos de dados.
- **2. Camada de Aplicação (`usecase/`):**
 - Orquestra a execução das regras de negócio e fluxos do jogo.
 - Declara as interfaces de repositórios necessárias para completar suas tarefas.
 - Executa validações de estado, aciona serviços e retorna DTOs tipados.
- **3. Camada de Infraestrutura (`repositories/`, `src/server/shared/`):**
 - Implementa os contratos de repositório utilizando tecnologias concretas: Drizzle ORM, PostgreSQL, Redis, APIs externas de IA.
 - Contém as definições de tabelas e serializadores técnicos.
- **4. Camada de Apresentação e Adaptação (`actions/`, `route.ts`):**
 - Converte a entrada bruta do mundo exterior (formulários FormData, parâmetros de rota, cookies HTTP).
 - Saneia os dados com esquemas Zod e despacha a execução para os Use Cases.
 - Mapeia o resultado e erros para a resposta padronizada de cliente.

12 Validação de Dados

12.1 O que é?

É a disciplina de verificação, transformação, validação de tipos e saneamento de todas as entradas de dados fornecidas pelo cliente ou por fontes externas antes de permitir qualquer processamento ou persistência no sistema. No projeto Olimpo, o motor canônico de validação de esquemas em tempo de execução e inferência estática é a biblioteca **Zod v4**.

12.2 Por que?

A máxima fundamental da segurança e confiabilidade de software é: *"Never trust user input"* (Nunca confie na entrada do usuário).

- **Prevenção contra Vetores de Ataque:** Validação rigorosa impede ataques de injeção (SQL Injection, Command Injection), estouro de tamanho de payloads, caracteres maliciosos e contaminação de protótipo.
- **Eliminação de Erros Silenciosos de Runtime:** Sem validação em tempo de execução, valores como `undefined`, `null` ou formatos inesperados podem propagar-se profundamente pelas camadas internas, corrompendo o banco de dados.
- **Ponte Segura entre TypeScript e Tempo de Execução:** O TypeScript realiza checagem de tipos estática que desaparece após a compilação; o Zod valida os dados reais no momento exato em que a requisição chega no servidor.

12.3 Como?

A validação de dados no Olimpo é implementada em múltiplas camadas de proteção:

1. **Validação de Borda em Server Actions com Zod:** Cada action define seu esquema estrito de entrada e processa os dados com `safeParse`:

```
const loginSchema = z.object({
  email: z.string().email("Endereço de e-mail inválido"),
  password: z.string().min(6, "A senha deve ter no mínimo 6 caracteres")
});

const parsed = loginSchema.safeParse(rawData);
if (!parsed.success) {
  return { success: false, error: parsed.error.issues[0]?.message };
}
```

2. **Inferência Automática de Tipos TypeScript:** Garantia de que os tipos TypeScript derivem automaticamente dos esquemas validados:

```
export type LoginActionInput = z.infer<typeof loginSchema>;
```

3. **Validação Intrínseca de Value Objects no Domínio:** Validações de regras invariantes do domínio (como o formato do PIN de 6 dígitos) são encapsuladas em métodos estáticos nos próprios Value Objects (ex.: `RoomPin.isValid(pin)`), garantindo que nenhum objeto inválido possa existir no domínio.
4. **Validação de Segurança de Rede (Anti-SSRF):** No motor de ingestão de notícias (`news/extract`), além de Zod, é utilizada a biblioteca `ipaddr.js` com resolução de DNS pinning via `undici` para validar e bloquear estritamente requisições direcionadas a faixas de IP privadas (`localhost`, `127.0.0.1`, `10.0.0.0/8`, `192.168.0.0/16`, etc.), neutralizando vulnerabilidades de SSRF (*Server-Side Request Forgery*).

13 Padrão de Código

13.1 O que é?

O **Padrão de Código** é o conjunto formal de convenções estilísticas, regras de tipagem estrita, ferramentas de análise estática de código (*linters*), formatadores determinísticos, convenções de commits e esteiras de verificação automática adotadas no projeto para garantir excelência de engenharia.

13.2 Por que?

Em um projeto de grande porte e com múltiplos colaboradores:

- **Legibilidade e Coerência Visual:** Um padrão unificado permite que qualquer desenvolvedor navegue e entenda o código instantaneamente, como se tivesse sido escrito por uma única pessoa.
- **Prevenção Preventiva de Bugs:** Regras estritas de linter identificam variáveis não utilizadas, dependências ausentes em hooks React e potenciais vulnerabilidades de concorrência antes mesmo do código ser executado.
- **Histórico de Versão Auditável:** O versionamento padronizado através de Conventional Commits simplifica a geração automática de changelogs e o rastreamento de defeitos (*bisect*).

13.3 Como?

O ferramental de governança de código do Olimpo é composto por:

- **TypeScript em Modo Estrito (`strict: true`):** Sem exceções permissivas, com flags ativas para desabilitar retornos implícitos e variáveis não verificadas.
- **ESLint 9 com Ecossistema Moderno:**

- `eslint-config-next` para melhores práticas de Next.js e React.
- `eslint-plugin-unused-imports` para remoção automática de imports órfãos.
- `eslint-plugin-jsx-a11y` para garantia de acessibilidade nas interfaces.
- **Prettier com Tailwind Plugin:** Formatação uniforme com largura de linha, aspas duplas, ponto-e-vírgula e o plugin `prettier-plugin-tailwindcss` para ordenação previsível e canônica de classes de estilo CSS.
- **Git Hooks com Husky e Lint-Staged:** A cada comando `git commit`, o Husky intercepta o processo e dispara o `lint-staged`, que formata e analisa apenas os arquivos alterados na *staging area*.
- **Conventional Commits com Commitlint:** Validação obrigatória da mensagem de commit seguindo o padrão da indústria:
 - `feat::` Nova funcionalidade.
 - `fix::` Correção de bug.
 - `docs::` Alterações exclusivamente em documentação.
 - `refactor::` Refatoração de código sem alteração de comportamento.
 - `test::` Adição ou correção de testes.
 - `chore::` Atualização de tarefas de build ou dependências.
- **Observabilidade e Logs Estruturados:** Emprego de **Pino** (`pino` e `pino-pretty`) no backend em vez de `console.log`, gravando registros estruturados em JSON enriquecidos com nível de severidade, timestamp e identificadores contextuais de requisição.

14 FSD (Feature-Sliced Design)

14.1 O que é?

O **Feature-Sliced Design (FSD)** é uma metodologia arquitetural moderna concebida especificamente para estruturar projetos de frontend complexos. Em oposição à abordagem tradicional baseada em tipos técnicos de arquivos (pastas como `/components`, `/hooks`, `/services`, que rapidamente se tornam monolíticas e cheias de referências circulares), o FSD decompõe o código visual da aplicação em camadas hierárquicas claras, fatias de negócio independentes e segmentos técnicos padronizados.

14.2 Por que?

Projetos frontend de larga escala frequentemente sofrem da chamada "crise de acoplamento", onde um componente simples importa acidentalmente regras de negócio globais ou serviços de outras telas, inviabilizando refatorações e testes:

- **Fluxo de Importação Controlado e Previsível:** O FSD impõe que uma camada só pode importar elementos de camadas que estejam hierarquicamente abaixo dela, erradicando totalmente dependências circulares.

- **Reutilização Coerente:** Componentes genéricos de UI permanecem puros e reutilizáveis, enquanto lógicas específicas de negócios ficam estritamente contidas em suas fatias temáticas.
- **Escalabilidade para Equipes Ágeis:** Múltiplos desenvolvedores podem atuar simultaneamente em diferentes *features* e *widgets* com mínimo risco de conflitos de merge no Git.

14.3 Como?

No ecossistema do Olimpo, o diretório `web-app/src/` adota o FSD distribuído nas seguintes camadas ordenadas (da mais baixa para a mais alta na hierarquia):

1. **Camada Base / Primitivas (`src/components/ui/`):** Blocos de construção atômicos e agnósticos a qualquer regra de negócio do Olimpo. Exemplos: botões estilizados, inputs de texto, cards, alertas, seletores de tema e ícones Lucide.
2. **Camada de Entidades do Frontend (`src/entities/`):** Representações conceituais do domínio projetadas para exibição na interface do usuário. Exemplo: `src/entities/news-article/` que contém os componentes visuais para renderização de cartões de notícia, badge de categorização e interfaces de dados exibíveis.
3. **Camada de Funcionalidades (`src/features/`):** Ações interativas completas do usuário que agregam valor direto ao negócio. Exemplo: `src/features/extract-news/`, encapsulando a caixa de entrada de URL de notícia, o botão de disparo, o tratamento do estado de loading e a invocação da Server Action correspondente.
4. **Camada de Widgets Compostos (`src/widgets/`):** Grandes blocos visuais autônomos que agrupam entidades e features para compor seções ricas da página. Exemplos: `src/widgets/app-header/` (barra de navegação superior com perfil e tema) e `src/widgets/news-ex` (painel completo com formulário e pré-visualização de resultados).
5. **Camada de Telas / Páginas (`src/views/`):** Montagem integral das telas do sistema integrando múltiplos widgets. Exemplos: `src/views/home/`, `src/views/olimp/` e `src/views/dev-sandbox/`.
6. **Camada de Aplicação do Next.js (`src/app/`):** Ponto de entrada do roteador Next.js App Router (`layout.tsx`, `page.tsx`, `globals.css`), responsável por injetar fontes, provedores globais de tema e mapear as URLs públicas para as respectivas *views*.

Hierarquia Estrita de Importações no Frontend:

`src/app` → `src/views` → `src/widgets` → `src/features` → `src/entities` → `src/components/ui`

15 Testes de Software e Garantia de Qualidade

15.1 O que é?

É a estratégia multidisciplinar e automatizada de verificação, validação e aferição de conformidade de código que permeia todas as camadas da plataforma Olimpo. Em vez de depender de testes manuais tardios ou inspecionar apenas a interface visual, o projeto adota uma **Pirâmide de Testes Abrangente** que cobre desde a menor unidade lógica e invariantes de domínio até a experiência completa do usuário em navegadores reais.

O ecossistema de testes do Olimpo é composto pelas seguintes modalidades:

- **Testes Unitários de Domínio e Casos de Uso:** Validação estrita de entidades puras e orquestração de negócios com **Vitest v4** em tempo de execução ultrarrápido (sub-segundo).
- **Testes de Integração de Repositórios e Serviços:** Verificação de persistência com **Drizzle ORM** e mensageria/cache com **Redis**, além de endpoints de streaming SSE.
- **Testes de Server Actions e Observabilidade:** Simulação do fluxo de ações mutáveis do Next.js com validação Zod, injeção de sessão e logs estruturados em JSON via Pino.
- **Testes de Componentes Visuais (UI):** Testes em isolamento de componentes React com **React Testing Library** e ambiente DOM emulado via **JSDOM**.
- **Testes Ponta a Ponta (End-to-End – E2E):** Simulação realista de jornadas de usuários em múltiplos motores de renderização (Chromium, Firefox e WebKit) através do **Playwright**.
- **Testes de Extração e Heurísticas de Notícias:** Validação do motor de extração de conteúdo com **@mozilla/readability**, fallback com Jina Reader API e segurança de rede anti-SSRF com **Undici** e **ipaddr.js**.

15.2 Por que?

Em um sistema gamificado e colaborativo com partidas multiplayer em tempo real e mecânicas de pontuação competitivas:

- **Prevenção de Regressões em Regras Críticas:** Pequenas alterações em cálculos de pontuação de blefe, contagem regressiva de rodadas ou transições de status de sala poderiam corromper partidas ativas.
- **Feedback Imediato para o Desenvolvedor (TDD):** A separação limpa com Injeção de Dependência permite que dezenas de suítes unitárias sejam executadas em menos de 2 segundos, acelerando o ciclo de desenvolvimento e permitindo refatorações com confiança absoluta.
- **Confiabilidade em Ambientes Serverless e Heterogêneos:** Testes E2E garantem que o empacotamento do Next.js, as hidratações de componentes React no cliente,

os cabeçalhos de cookies e os streams SSE funcionem em harmonia antes de qualquer implantação em produção.

- **Conformidade Contínua e Governança Técnica:** A validação unificada garante que código quebrado, com erros de tipo ou sem testes seja bloqueado antes mesmo do merge no repositório.

15.3 Como?

A estrutura de testes no repositório é organizada e executada conforme a taxonomia abaixo:

1. Testes Unitários de Domínio e Casos de Uso (`*.usecase.test.ts`, `*-entity.test.ts`):

- Localizados co-localizados dentro de cada funcionalidade em `src/app/api/{feature}/tests/`.
- Utilizam dublês de teste (mocks de repositórios em memória implementando interfaces como `IUserRepository`, `IRoomRepository` ou `IAIAnalysisService`).
- Exemplos reais: `login.usecase.test.ts`, `create-room.usecase.test.ts`, `submit-vote.usecase.test.ts`, `ai-analysis-entity.test.ts`.
- Comando: `pnpm test:unit` (executa via Vitest com compilação instantânea).

2. Testes de Integração de Infraestrutura e Repositórios:

- Validam operações reais de leitura, escrita e queries Drizzle (`drizzle-room.repository.test.ts`, `drizzle-news-vote.repository.test.ts`).
- Validam comandos em memória e publicações de canal no Redis (`redis-room.repository.test.ts`, `redis-vote.repository.test.ts`, `redis-event.publisher.test.ts`).
- Validam o ciclo de vida do stream SSE (`sse-route.test.ts`), garantindo que o fechamento da conexão acione o cancelamento do subscriber.

3. Testes de Server Actions e Observabilidade (`*-actions.test.ts`, `*.e2e.test.ts`):

- Validam a resposta discriminada (`success: true/false`), rejeição de entradas maliciosas com mensagens amigáveis e gravação correta de cookies no cabeçalho.
- Integração com logs estruturados do Pino testados em `action-test-logger.test.ts`.

4. Testes de Componentes Frontend (`web-app/tests/components/`):

- Utilizam `@testing-library/react`, `@testing-library/jest-dom` e `@testing-library/user-event`.
- Exemplos: `extract-news-form.test.tsx` e `article-preview.test.tsx`, simulando cliques, digitação de URLs, exibição de spinners de carregamento e estados de erro acessíveis.

5. Testes End-to-End com Playwright (`web-app/tests/e2e/`):

- Configurados em `playwright.config.ts`.
- Inicializam o servidor local Next.js e exercitam fluxos completos de tela (`home.spec.ts`).
- Comandos: `pnpm test:e2e` (modo headless para CI) e `pnpm test:e2e:ui` (modo interativo com depuração visual e time-travel).

6. Testes de Pipeline de Notícias e Heurísticas de Segurança (web-app/tests/):

- Executados com `tsx -test: news.test.ts, jina-text.test.ts, plain-content.test.ts`.
- Validam a extração de manchetes reais, conversão HTML para Markdown e bloqueio de SSRF contra IPs privados via `ipaddr.js`.

7. Cobertura de Código e Validação Completa:

- Cobertura de código via V8 engine: `pnpm test:coverage`.
- Pipeline de validação unificada pré-commit / pré-deploy:

```
pnpm validate ≡ pnpm lint && pnpm typecheck && pnpm test && pnpm build
```

16 Considerações Finais

A arquitetura do **Olimpo — Fake News AI** foi projetada para garantir alto desempenho, segurança e manutenibilidade a longo prazo. A integração entre governança técnica (**OpenSpec**), desacoplamento de regras de negócio (**Clean Architecture**), persistência híbrida de baixa latência (**PostgreSQL** no **Supabase** e **Redis**), eventos em tempo real via **SSE** e organização modular (**FSD**) estabelece uma base robusta e escalável para o sistema.

A evolução contínua da aplicação deve priorizar a pureza do domínio de negócio, a validação rigorosa de entradas de rede e a cobertura contínua de testes automatizados, sustentando uma experiência multiplayer estável, interativa e segura no combate à desinformação.

Equipe de Engenharia de Software — Olimpo AI
PUCC & Instituto de Pesquisas Eldorado